
第 2 章

Sympy 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

2. SymPy 及其基本使用（新版讲义）

本页是第 2 章的章首页（索引）。正文位于 01-符号对象与基本运算.md ~ 04-常见误区与技巧.md，与原版资料完全分离（原版见 ../../原始资料/chap2/）。

本章概览

SymPy 是 Python 的符号计算（symbolic computation）核心库：它不计算具体数值，而是把 x 、 y 、 π 当作符号进行解析推导，从而可以精确地求极限、导数、积分、化简、解方程与微分方程、处理符号矩阵，并给出数学上的精确解。与 NumPy 的数值解相比，SymPy 能保留 $\frac{1}{3}$ 、 $\sqrt{2}$ 这样的精确形式，也更能反映数学本质。

本章从“如何创建符号对象”开始，逐步过渡到“用 SymPy 求解数学问题”：方程、方程组、微分方程、符号矩阵，最后用综合案例把符号推导与数值验证、可视化串起来。

学习目标

学完本章，你应该能够：

1. 创建并理解符号对象（symbols / Symbol），会使用 positive、real、integer 等假设。
2. 用 SymPy 完成符号的算术运算、表达式化简（simplify / expand / factor / trigsimp）与替换求值（subs / evalf / N）。
3. 求符号极限（limit）、导数与偏导数（diff）、不定/定积分与二重积分（integrate）。
4. 解方程与方程组：solve、solveset、linsolve、nonlinsolve，并理解 Eq 与“表达式 = 0”的约定。
5. 用 Symbolic Matrix 做矩阵运算、特征值、雅可比矩阵，与 NumPy 数值结果互相验证。
6. 用 lambdify 把符号表达式转换为可调用的 NumPy 函数，实现“符号推导 → 数值计算”的衔接。
7. 用 dsolve 求解一阶/二阶常微分方程及初值问题，并画出解的曲线。
8. 完成一个综合小案例（如符号求面积 + 可视化、符号导数 + 切线图），体会“精确推导 + 数值验证 + 可视化”的完整流程。

案例索引

案例卡	所在节	技术点	难度
1. 符号与数值的区别——为什么 x 不是 1.0	01 符号对象与基本运算	Symbol、subs、evalf、符号 vs 数值	★☆☆
2. 用 solve/solveset 解方程并判断根的类型	02 利用 SymPy 求解问题	Eq、solve、solveset、domain、根类型	★★☆
3. lambdify 把公式变成可计算函数	02 利用 SymPy 求解问题	lambdify、numpy、向量化	★★☆
4. 符号积分求面积并数值验证	03 综合案例	符号积分、N、matplotlib	★★☆
5. 符号导数 → 切线方程 → 数值验证	03 综合案例	diff、subs、lambdify、差商	★★☆
6. 解方程并把根画在数轴上	03 综合案例	solve、根可视化	★★☆

先修要求与运行环境

- 熟悉 Python 基础语法（变量、函数、循环；可先学 聪明办法学 Python）。
- 了解基本的微积分与线性代数概念（极限、导数、积分、矩阵、特征值）。

- 安装 Python 3.10+ 与 SymPy（同时建议安装 NumPy 与 Matplotlib，便于数值验证与画图）：

```
pip install sympy
# 若需要数值验证/可视化，再加装
pip install numpy matplotlib scipy pandas
```

本机已验证：Python 3.13.5, SymPy 1.13.1, NumPy 2.1.2, Matplotlib 3.10.1。统计学建模库 statsmodels 0.14.6 也已可用（本章不使用，后续章节再用）。

本章目录

小节	文件	内容
01 符号对象与基本运算	01-符号对象与基本运算.md	符号创建、假设、算术、函数、极限、导数、积分、化简与替换、1 个精讲案例卡
02 利用 SymPy 求解问题	02-利用 SymPy 求解问题.md	Eq、solve/solveset/linsolve/nonlinsolve、符号矩阵、lambdify、dsolve、2 个精讲案例卡
03 综合案例	03-综合案例.md	符号求面积 + 可视化、符号导数 + 切线图、根的可视化（图文案例）
04 常见误区与技巧	04-常见误区与技巧.md	易错点清单、性能技巧、调试建议

数学预备与附录

本章的微积分/方程背景已集中到附录《数学与算法补充》：

- **B 微积分与数值方法**：B.1 微积分直觉、B.2 关键公式速查、B.3.5 常微分方程；
- **A 线性代数与矩阵**：A.3 特征值分解（SymPy 解特征问题）；
- [打开附录 B](#) | [附录索引](#)

做极限/导数/积分/方程求解前，先读 B.1 与 B.2 的公式表。

练习与上机入口

- [本章练习 \(exercises/\)](#)：作业题、自测 quiz、参考答案（从历史 chapter_2_3_quiz 整理出 SymPy 部分）。
- [本章上机 \(lab/\)](#)：循序渐进的上机 notebook，含综合任务与检查清单。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题集、中文资料。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

环节	学时	对应内容
讲课	3–4 学时	01–02 正文 + 03 案例讲解
上机	2–3 学时	lab/ 逐题完成；课后完成 exercises/ 作业

使用说明

- **学生**：先读 01-02 正文并运行代码 → 完成 lab 上机 → 提交 exercises 作业 → 自测 quiz 检验。生词/公式不懂时，到 references.md 找官方对应章节。
- **教师**：按 teaching.md 的课时表讲；lab 可作为上机课内容；exercises 中的 quiz 带自动评分，可直接回收。

01 符号对象与基本运算：创建、算术、极限、微分与积分

本节对应原版 2.1 的内容，并增补学习目标、常见误区、思考题与延伸阅读。所有代码输出均为实际运行结果 (SymPy 1.13.1)。

本节目标

- 理解 “符号对象 (symbolic object)” 与 “数值对象” 的区别；
- 会用 `symbols / Symbol` 创建符号，并设置 `positive`、`real`、`integer` 等假设；
- 掌握符号的四则运算、常用数学函数 (`sin`、`cos`、`exp` 等)；
- 会求极限 (`limit`)、导数/偏导数 (`diff`)、不定积分与定积分 (`integrate`)；
- 会用 `simplify`、`factor`、`expand`、`trigsimp` 化简，用 `subs / evalf / N` 做替换与数值化；
- 了解 `series` (级数展开) 等进阶工具。

先修

- Python 变量、函数、字符串；
- 微积分中的极限、导数、积分概念 (不要求会手算复杂积分)。

官方文档/参考入口

- SymPy 官方 Tutorial (符号、化简、微积分): [链接](#)
- SymPy 官方 Calculus: [链接](#)

1.1 为什么用 SymPy

NumPy 擅长**数值**计算：把数字代入后算出一个近似值。SymPy 则进行**符号**计算：把 x 、 y 当作字母，推导出精确表达式。

特性	NumPy (数值)	SymPy (符号)
代表对象	<code>ndarray</code>	<code>Symbol / Expr</code>
$\int_0^1 x^2 dx$	返回 0.3333333333	返回精确的 $1/3$
解方程	数值迭代	精确根 $\sqrt{2}$ 等
用途	大规模数值模拟	公式推导、理论验证

实际工作中二者常配合使用：先用 SymPy 推导出公式，再 `lambdify` 成数值函数 (见 02 节) 交给 NumPy 计算。

1.2 符号对象的创建

1.2.1 一次创建多个符号: `symbols`

```
import sympy as sp
x, y, z = sp.symbols('x y z')
print(x + y + z)
x + y + z
```

1.2.2 单个符号: Symbol

```
x = sp.Symbol('x')
print(x)

positive_x = sp.Symbol('x', positive=True)
print(positive_x)

x
x
```

注意: `positive=True` 并不改变打印结果, 但会让 SymPy 在化简时利用 “ $x > 0$ ” 这个信息 (例如 $\sqrt{x^2}$ 会化简成 x)。

其他常见假设: `real` (实数)、`integer` (整数)、`complex` (复数)。

```
real_x = sp.Symbol('x', real=True)
integer_n = sp.Symbol('n', integer=True)
print(real_x)
print(integer_n)

x
n
```

1.2.3 创建一系列相关符号

```
n = 3
xs = sp.symbols(f'x_{1}:{n+1}')
print(xs)

x_list = [sp.Symbol(f'x_{i+1}') for i in range(3)]
print(x_list)

(x_1, x_2, x_3)
[x_1, x_2, x_3]
```

1.3 符号对象的算术运算与常用函数

1.3.1 四则运算

```
x, y = sp.symbols('x y')
print(x + y)
print(x - y)
print(x * y)
print(x / y)

x + y
x - y
x*y
x/y
```

1.3.2 符号函数

用 Function 可以定义“抽象函数”，但更常见的是直接使用 SymPy 预定义函数 (sin、cos、exp、log、sqrt 等)。

```
x = sp.symbols('x')
f = sp.Function('f')(x)
print(f)
print(f.diff(x))          # 抽象函数的导数

sin_x = sp.sin(x)
cos_x = sp.cos(x)
print(sin_x)
print(cos_x)
print(sin_x + cos_x)

f(x)
Derivative(f(x), x)
sin(x)
cos(x)
sin(x) + cos(x)
```

1.4 极限运算

limit(expression, variable, point, dir='+') 的四个参数分别是被求极限的表达式、变量、趋近点、方向 ('+' 右极限、 '-' 左极限、 '+-' 双侧)。oo 表示无穷大。

```
x = sp.symbols('x')
print(sp.limit(sp.sin(x)/x, x, 0))          # 重要极限 -> 1
print(sp.limit(1/x, x, sp.oo))              # x 趋近 +oo -> 0
print(sp.limit(1/x, x, 0, dir='-'))         # 左极限 -> -oo
print(sp.limit(x**3 + 3*x**2 + sp.exp(-2*x), x, 0.5))

1
0
-oo
1.24287944117144
```

1.5 微分运算

1.5.1 一元函数求导

```
x = sp.symbols('x')
f = x**2 + 3*x + 2
print(sp.diff(f, x))

g = sp.sin(x)*sp.cos(x)
print(sp.diff(g, x))
print(sp.simplify(sp.diff(g, x)))          # 化简为 cos(2x)
```

```
2*x + 3
-sin(x)**2 + cos(x)**2
cos(2*x)
```

1.5.2 多元函数求偏导

```
x, y = sp.symbols('x y')
h = x**2 + x*y + y**2
print(sp.diff(h, x))
print(sp.diff(h, y))

2*x + y
x + 2*y
```

1.5.3 高阶导数

```
x = sp.symbols('x')
f = x**3
print(sp.diff(f, x, 2))    # 二阶导数

6*x
```

1.6 积分运算

1.6.1 不定积分

```
x = sp.symbols('x')
f = x**2 + 2*x + 1
print(sp.integrate(f, x))

x**3/3 + x**2 + x
```

1.6.2 定积分

```
x = sp.symbols('x')
print(sp.integrate(x**2, (x, 0, 1)))    # 精确的 1/3

1/3
```

1.6.3 二重积分 (嵌套调用)

```
x, y = sp.symbols('x y')
f = x + y
print(sp.integrate(sp.integrate(f, (x, 0, 1)), (y, 0, 1)))

1
```

SymPy 没有单独的“重积分”函数，通过嵌套 `integrate` 即可实现；曲线/曲面积分一般需要先做参数化再代入。

1.7 表达式化简、替换与数值化

1.7.1 化简与展开

```
x = sp.symbols('x')
expr = x**2 + 2*x + 1
print(sp.factor(expr))          # 因式分解
print(sp.expand((x+1)**2))      # 展开
print(sp.simplify(sp.sin(x)**2 + sp.cos(x)**2))  # 恒等于 1
print(sp.trigsimp(sp.sin(x)**2 + sp.cos(x)**2))

(x + 1)**2
x**2 + 2*x + 1
1
1
```

1.7.2 替换与数值化

```
x = sp.symbols('x')
print(sp.sqrt(8))               # 精确形式
print(sp.sqrt(8).evalf())       # 数值
print((x**2 + 2*x + 1).subs(x, 2))
print(sp.N(sp.sqrt(8)))         # 与 evalf 等价

2*sqrt(2)
2.82842712474619
9
2.82842712474619
```

1.7.3 级数展开

```
x = sp.symbols('x')
print(sp.series(sp.sin(x), x, 0, 6))

x - x**3/6 + x**5/120 + O(x**6)
```

案例卡 1：符号与数值的区别——为什么 x 不是 1.0

目标

用 SymPy 创建符号 x 之后， x 是一个**符号对象**，不是具体的数字。这个案例让你亲眼看到：打印 $x + 1$ 得到的是表达式 $x + 1$ ，而不是某个数值；只有用 `subs` 代入具体数字、再用 `evalf/N` 数值化，才会得到普通浮点数。理解了这一点，你以后才不会把 SymPy 当成“升级版计算器”误用。

代码

```
import sympy as sp
import numpy as np
np.set_printoptions(precision=4, suppress=True)

x = sp.Symbol('x')
print('type(x) =', type(x).__name__)          # Symbol
```

```

print('x + 1 =', x + 1)           # 仍是表达式
print('x * 2 =', x * 2)           # 2*x
print('x == 1.0:', x == 1.0)      # 符号不等于数字
print('x + 1 == 2:', x + 1 == 2)  # 结构不同

# 代入具体值: subs 返回 “新的表达式”, 不会改动 x
v = (x**2 + 2*x + 1).subs(x, 3)
print('subs x=3 后:', v, ' type:', type(v).__name__) # 16 Integer
print('evalf:', sp.N(v))                        # 16.000000000000000
print('float:', float(sp.N(v)))                  # 16.0

# 符号 pi 与浮点 pi 的区别
print('sp.pi =', sp.pi)                  # pi
print('float(sp.pi) =', float(sp.pi))      # 3.141592653589793
print('sp.sin(sp.pi/2) =', sp.sin(sp.pi/2)) # 精确的 1
import math
print('math.sin(math.pi/2) =', math.sin(math.pi/2)) # 浮点近似 1.0

```

运行结果 (已运行核验)

```

type(x) = Symbol
x + 1 = x + 1
x * 2 = 2*x
x == 1.0: False
x + 1 == 2: False
subs x=3 后: 16 type: Integer
evalf: 16.000000000000000
float: 16.0
sp.pi = pi
float(sp.pi) = 3.141592653589793
sp.sin(sp.pi/2) = 1
math.sin(math.pi/2) = 1.0

```

讲解

一句话解释: SymPy 里的 x 是一张“写有 x 的纸条”，你可以对它做代数运算，但要得到数字，必须把具体的数写上去 (subs) 再“读出数值” (evalf)。

- `sp.Symbol('x')` 创建符号。 `type(x).__name__` 打印 `Symbol`，说明它是一个符号对象，不是 `float` 或 `int`。
- $x + 1$ 、 $x * 2$ 结果分别是 $x + 1$ 、 $2*x$ ——SymPy 把 $+$ 、 $*$ 当成**表达式构造**，而不是浮点加法。所以不会算出 2.0、3.0 之类的数字。
- $x == 1.0$ 为 `False`：== 比较的是“是不是同一个结构/对象”，而符号 x 与数字 1.0 结构不同。要判断“符号表达式是否数学等价”，用 `simplify(a - b) == 0` 或 `a.equals(b)` (见 04 小节)。
- `(x**2 + 2*x + 1).subs(x, 3)` 把 x 换成 3，SymPy 先算出**精确整数** 16 (类型 `Integer`)。 `sp.N(v)` 把它数值化成 `16.000000000000000`， `float(sp.N(v))` 得到 Python 的浮点 16.0。
- `sp.pi` 是符号 π ； `sp.sin(sp.pi/2)` 精确返回 1。 `math.sin(math.pi/2)` 是“先用浮点近似 π 再取正弦”，得到 1.0 (浮点)，两者在不要求精确时看起来一样，但在符号推导里 `sp.pi` 更“干净”。

主要用法 / API

需求	写法	返回
创建单个符号	<code>sp.Symbol('x')</code>	Symbol
创建多个符号	<code>sp.symbols('x y')</code>	元组
替换变量	<code>expr.subs(x, 3)</code>	新的表达式
符号表达式数值化	<code>sp.N(expr) / expr.evalf()</code>	浮点
转成 Python 浮点	<code>float(sp.N(expr))</code>	float
符号常量	<code>sp.pi</code> 、 <code>sp.oo</code>	符号

常见错误

- 把 $x + 1$ 当成 “一个会自动算数值的表达式”，其实它是符号对象；
- 用 `==` 比较 $x + 1$ 与 $x + 1$ 之外的等价形式 (如 $\sin(x)^2 + \cos(x)^2$ 与 1)，得到 False；
- 想得到数值却忘了 `subs` 或 `evalf`，直接打印表达式；
- 用 `math.pi` 参与符号运算导致浮点污染。

拓展 / 跟练

1. 试着 `sp.simplify(x + 1 - 1)`，看结果是什么？为什么？
2. 给 `x = sp.Symbol('x', positive=True)` 后，`sp.sqrt(x**2)` 与不设假设时有什么不同？
3. 用 `sp.Rational(1, 3)` 和 `1/3` 分别做 `sp.N`，观察精度差异。
4. 用 `sp.pi` 与 `math.pi` 各自计算 $\sin(\pi)$ ，比较是否都是 0。

常见误区

1. 把 **SymPy 当 NumPy 用**： `x = Symbol('x')` 后 $x + 1$ 是符号表达式，不是数值；要数值化用 `evalf` / `N` / `lambdify`。
2. **忘记导入**： `sympy.symbols` 需要 `from sympy import symbols` 或 `import sympy as sp`。
3. **误用 Python 的 pi**： SymPy 用 `sp.pi` (符号 π)，不是 `math.pi` (浮点近似)。
4. **以为 simplify 一定能化简**： 它是 “尽力而为”，有时需要 `trigsimp`、`radsimp`、`cancel` 等配合。
5. **混淆 subs 与赋值**： `expr.subs(x, 2)` 返回新表达式，不会修改 `expr`。
6. **用 == 判断符号表达式相等**： 应使用 `sympy.simplify(a - b) == 0` 或 `a.equals(b)`；两个形式不同但数学等价 (如 $\sin(x)^2 + \cos(x)^2$ 与 1) 的表达式用 `==` 会得到 False。

思考题

1. `sp.sqrt(x**2)` 在没有假设时返回什么？加上 `x = Symbol('x', positive=True)` 后呢？为什么？
2. `sp.Rational(1, 3)` 与 `1/3` 有什么不同？各在何时有用？
3. 如何用 `diff` 求三元函数 $f(x, y, z) = x^2y + yz + z^2$ 对 y 的偏导？
4. `sp.integrate(sp.sin(x), (x, 0, sp.oo))` 是否存在？用 SymPy 验证。

动手练习（详见 lab）

- 创建 $x_1, \dots, x_5$ 五个符号并求和；
- 求 $f(x) = \frac{\sin x}{x}$ 在 0 处极限、导数、在 $[0, \pi]$ 上的定积分；
- 把 $\cos(2x)$ 用 `expand_trig` 展开，再用 `trigsimp` 还原；
- 用 `latex` 把表达式转成 LaTeX 字符串（进阶）。

延伸阅读

- SymPy 官方 Tutorial (Symbols、Algebraic manipulations、Calculus): [链接](#)
- SymPy 官方 Simplification 文档: [链接](#)
- SymPy 官方 Calculus 文档: [链接](#)

02 利用 SymPy 求解问题：方程、矩阵、lambdify 与微分方程

本节对应原版 2.2 的内容，并增补学习目标、常见误区、思考题与延伸阅读。所有代码输出均为实际运行结果 (SymPy 1.13.1)。

本节目标

- 会用 Eq 表示等式，理解“把表达式当作等于 0”的约定；
- 掌握 solve、solveset、linsolve、nonlinsolve 解方程与方程组；
- 会用 Symbolic Matrix 做矩阵运算、求特征值、计算雅可比矩阵；
- 会用 lambdify 把符号表达式转成可调用的 NumPy 函数，实现符号 $\rightarrow$ 数值的对接；
- 会用 dsolve 求一阶、二阶常微分方程的通解与初值解。

先修

- 01 节全部内容 (符号创建、极限、微分、积分)；
- 线性代数基础 (矩阵乘法、行列式、逆、特征值)。

官方文档/参考入口

- SymPy 官方 Solvers: [链接](#)
- SymPy 官方 lambdify: [链接](#)
- SymPy 官方 ODE: [链接](#)

2.1 用 SymPy 解方程

2.1.1 用 Eq 表示等式

```
import sympy as sp
x = sp.symbols('x')
equation = sp.Eq(x**2, 4)
print(sp.solveset(equation, x))
{-2, 2}
```

如果不写 Eq，直接把表达式传给求解函数，SymPy 默认认为该表达式等于 0：

```
x = sp.symbols('x')
print(sp.solveset(x**2 - 4, x))
{-2, 2}
```

2.1.2 solve 解一元方程

```
x = sp.symbols('x')
print(sp.solve(3*x + 4 - 10, x))          # 3x+4=10 -> x=2
print(sp.solve(x**2 + 5*x + 4, x))        # x^2+5x+4=0 -> -4, -1

[2]
[-4, -1]
```

2.1.3 solveset 与解集/复数域

```
x = sp.symbols('x')
print(sp.solveset(x**2 - 5*x + 6, x))           # {2,3}
print(sp.solveset(2*x**2 + 4*x + 2, x))         # 重根
print(sp.solveset(x**2 + 1, x, domain=sp.S.Complexes)) # 复数域

{2, 3}
{-1}
{-I, I}
```

函数	返回	特点
solve	列表 [...]	传统易读，适合大多数方程
solveset	集合 {...}	解集表达更严谨，支持 domain

2.1.4 解方程组

解 $\begin{cases} 2x + 3y = 5 \\ 4x - 2y = 10 \end{cases}$:

```
x, y = sp.symbols('x y')
eqs = [2*x + 3*y - 5, 4*x - 2*y - 10]
print(sp.solve(eqs, (x, y)))

{x: 5/2, y: 0}
```

解非线性方程组 $\begin{cases} x^2 + y^2 = 5 \\ x - y = 1 \end{cases}$:

```
x, y = sp.symbols('x y')
eqs = [x**2 + y**2 - 5, x - y - 1]
print(sp.solve(eqs, (x, y)))

[(-1, -2), (2, 1)]
```

复杂的非线性方程组可能返回复数解、RootOf 对象或空列表，需要结合题目与 domain 参数判断。

2.1.5 linsolve 解线性方程组

```
x, y, z = sp.symbols('x y z')
eqs = [x + y + z - 2, 2*x - y + z + 1, x + 2*y + 2*z - 3]
print(sp.linsolve(eqs, (x, y, z)))

{(1, 2, -1)}
```

也可以用系数矩阵 A 与常数向量 b :

```
A = sp.Matrix([[1, 1, 1], [2, -1, 1], [1, 2, 2]])
b = sp.Matrix([2, -1, 3])
print(sp.linsolve((A, b), (x, y, z)))

{(1, 2, -1)}
```

2.1.6 nonlinsolve 解非线性方程组

```
x, y = sp.symbols('x y')
eqs = [x**2 + y**2 - 2, x**3 + y**3]
sols = sp.nonlinsolve(eqs, (x, y))
real = sorted([s for s in sols if sp.im(s[0]) == 0 and sp.im(s[1]) == 0], key=str)
print('实数解:', real)
print('解集个数:', len(sols))
```

实数解: $[(-1, 1), (1, -1)]$

解集个数: 6

nonlinsolve 原样返回 6 个解 (2 个实数、4 个复数); 上面用 $\text{im}(s[0]) == 0$ and $\text{im}(s[1]) == 0$ 过滤出两个实数解。实际题目若只关心实数解, 可照此筛选。

案例卡 2: 用 solve/solveset 解方程并判断根的类型

目标

同一个方程, solve 与 solveset 会返回不同形式; 而且在实数域或无解/重根/复数根情况下, 结果差异很大。这个案例让你学会: 先选对求解函数, 再根据 domain 判断根是实数还是复数、是单根还是重根、是有理根还是无理根。

代码

```
import sympy as sp

x = sp.symbols('x', real=True)

# 一元二次方程  $x^2 - 5x + 6 = 0$ 
eq = sp.Eq(x**2 - 5*x + 6, 0)
print('solve      :', sp.solve(eq, x))
print('solveset   :', sp.solveset(eq, x))

# 重根  $(x-2)^2 = 0$ 
print('solve 重根   :', sp.solve((x - 2)**2, x))
print('solveset 重根 :', sp.solveset((x - 2)**2, x))

# 实数域与复数域
print('x^2+1 Reals   :', sp.solveset(x**2 + 1, x, domain=sp.S.Reals))
print('x^2+1 Complex :', sp.solveset(x**2 + 1, x, domain=sp.S.Complexes))

# 判断根的类型
r1 = sp.Rational(1, 2)
r2 = sp.sqrt(2)
print('有理根 r1 =', r1, ' 无理根 r2 =', r2)
print('r1.is_rational =', r1.is_rational, '  r2.is_rational =', r2.is_rational)
```

运行结果（已运行核验）

```
solve      : [2, 3]
solveset   : {2, 3}
solve 重根  : [2]
solveset 重根 : {2}
x^2+1 Reals   : EmptySet
x^2+1 Complex : {-I, I}
有理根 r1 = 1/2 无理根 r2 = sqrt(2)
r1.is_rational = True   r2.is_rational = False
```

讲解

一句话解释： solve 爱把结果塞进一个列表里，solveset 更喜欢用集合表示，并且能告诉你“这个根只在复数里出现”这类信息——所以做题时要看清你要的是“一个解”还是“所有解”，以及“是不是实数解”。

- `sp.Eq(x**2 - 5*x + 6, 0)` 明确写出“左边等于右边”。`solve` 返回列表 `[2, 3]`，`solveset` 返回集合 `{2, 3}`。两者数值一样，但形式不同：列表适合按顺序取，集合自动去重且数学上更严谨。
- 对重根 $(x-2)^2$ ，`solve` 返回 `[2]`，`solveset` 也返回 `{2}`。这两个函数都不会把“重数”写出来；若需要计算重根次数，要看 `roots` 或 `Poly` 的 `multiplicity`（本节先知道“有重根”即可）。
- $x^2+1=0$ 在**实数域** `sp.S.Reals` 上没有解，所以 `solveset` 返回 `EmptySet`；换成**复数域** `sp.S.Complexes` 就得到 `{-I, I}`。这正是 `solveset` 的 `domain` 参数的价值：帮你把“题目要求实数解”限定清楚。
- `sp.Rational(1, 2)` 是精确的有理数 `1/2`，其 `.is_rational` 为 `True`；`sp.sqrt(2)` 是无理根，`.is_rational` 为 `False`。用这些 `.is_xxx` 属性可以程序化地判断根的类型。

主要用法 / API

需求	写法	返回
解一元方程（列表形式）	<code>sp.solve(eq, x)</code>	<code>[...]</code>
解一元方程（集合形式）	<code>sp.solveset(eq, x)</code>	<code>FiniteSet / EmptySet</code>
限定实数域	<code>sp.solveset(f, x, domain=sp.S.Reals)</code>	集合
限定复数域	<code>sp.solveset(f, x, domain=sp.S.Complexes)</code>	集合
判断是否为有理数	<code>root.is_rational</code>	<code>True / False</code>
精确分数	<code>sp.Rational(1, 2)</code>	<code>Rational</code>

常见错误

- 以为 `solve` 与 `solveset` 永远等价：重根、无解、复数域场景下返回形式不同；
- 求实数根却忘了 `domain=sp.S.Reals`，结果里混入 I （虚数单位）；
- 用 `==` 判断“某个数是不是解”：应先 `eq.subs(x, r)` 再 `sp.simplify(result) == 0`；
- 把 `sp.solve(f, x)` 中 `f` 直接写成 `f = 0` 的字符串（如 `'x**2-4'`），那是字符串不是表达式。

拓展 / 跟练

1. 解 $x^3 - 6x^2 + 11x - 6 = 0$ ，分别用 `solve` 与 `solveset` 比较；
2. 用 `sp.solveset(x**2 + 1, x, domain=sp.S.Reals)` 看空集，再解释为什么；

3. 用 `root.is_integer / root.is_real` 判断 $\sqrt{4}$ 是否整数 (提示: 先化简)。

2.2 用 Symbolic Matrix 做矩阵运算

2.2.1 基本运算

```
A = sp.Matrix([[1, 2], [3, 4]])
B = sp.Matrix([[5, 6], [7, 8]])
print(A + B)
print(A * B)
print(A.det())
print(A.inv())
print(A.solve(sp.Matrix([1, 2])))
print(A**2)

Matrix([[6, 8], [10, 12]])
Matrix([[19, 22], [43, 50]])
-2
Matrix([[-2, 1], [3/2, -1/2]])
Matrix([[0], [1/2]])
Matrix([[7, 10], [15, 22]])
```

2.2.2 特征值

```
A = sp.Matrix([[2, 1], [1, 2]])
print(A.eigenvals())
print(A.eigenvects())

{3: 1, 1: 1}
[(1, 1, [Matrix([
[-1],
[ 1]])]), (3, 1, [Matrix([
[1],
[1]])])]
```

2.2.3 雅可比矩阵

```
x, y = sp.symbols('x y')
f1 = x**2 + y
f2 = sp.sin(x) - y**2
J = sp.Matrix([f1, f2]).jacobian([x, y])
print(J)

Matrix([[2*x, 1], [cos(x), -2*y]])
```

2.3 用 lambdify 把符号表达式变成数值函数

`lambdify` 是 SymPy 与 NumPy 之间的“桥梁”：把符号表达式编译成可调用的 Python/NumPy 函数，便于对大数组做高效数值计算。

```
import sympy as sp
x = sp.symbols('x')
expr = x**2 + 2*x + 1
f = sp.lambdify(x, expr, 'numpy')
print(f(1))
print(f(2))
f_basic = sp.lambdify(x, expr)
print(f_basic(1))
4
9
4
```

传入 NumPy 数组:

```
import sympy as sp, numpy as np
x = sp.symbols('x')
f = sp.lambdify(x, sp.sin(x)**2, 'numpy')
print(f(np.array([0, np.pi/2, np.pi])))
[0.00000000e+00 1.00000000e+00 1.49975978e-32]
```

注意: lambdify 生成的是数值函数, 返回的是 NumPy 数组或标量; f(1) 返回 4 (不是原档里误写的 3)。

案例卡 3: lambdify 把公式变成可计算函数

目标

SymPy 给出的是一张“公式纸条”, 不能直接对数组做高效计算。lambdify 就是那座桥: 把符号表达式编译成一个真正能接收数字/NumPy 数组的 Python 函数, 从而把“符号推导”和“数值计算/画图”无缝接起来。

代码

```
import sympy as sp
import numpy as np
np.set_printoptions(precision=4, suppress=True)

x, y = sp.symbols('x y')
f = x**2 + 2*x*y + y**2 # 符号公式: (x+y)^2
f_num = sp.lambdify((x, y), f, 'numpy')

print('lambdify(x,y) 类型:', type(f_num).__name__)
print('f_num(1, 2) =', f_num(1, 2)) # (1+2)^2 = 9
print('f_num(3, 4) =', f_num(3, 4)) # (3+4)^2 = 49

# 一次性算一堆点 (向量化): 这就是“符号 -> 数值”的价值
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print('向量化 f_num(a, b) =', f_num(a, b)) # [25 49 81]
print('用 subs 逐个验证 =', [f.subs({x: i, y: j}) for i, j in zip(a, b)])
```

运行结果（已运行检验）

```
lambdify(x,y) 类型: function
f_num(1, 2) = 9
f_num(3, 4) = 49
向量化 f_num(a, b) = [25 49 81]
用 subs 逐个验证 = [25, 49, 81]
```

讲解

一句话解释：lambdify 把“带 x、y 的公式” 变成一个“填数字就出结果” 的普通函数，而且它能一口气处理整个 NumPy 数组，不用你写 for 循环。

- `sp.lambdify((x, y), f, 'numpy')` 第一个参数是符号（单个符号或符号元组），第二个是表达式，第三个 'numpy' 告诉它“生成的函数要能处理 NumPy 数组”。
- `f_num(1, 2)` 返回 9：因为 $f = (x + y)^2$ ，代入 (1, 2) 得 $3^2 = 9$ 。`f_num(3, 4)` 同理得到 49。
- `f_num(a, b)` 中 a、b 是长度 3 的 NumPy 数组，lambdify 生成的是**向量化**函数，一次算出 [25, 49, 81]。这是它比“循环里逐点 subs / evalf” 快很多的原因。
- 最后用 subs 逐个代入验证：[25, 49, 81]，与 lambdify 结果一致，说明二者数学上等价。注意 subs 返回的是符号/整数对象，lambdify 返回的是普通数值/NumPy 数组。

主要用法 / API

需求	写法	返回
一元符号表达式转数值函数	<code>sp.lambdify(x, expr, 'numpy')</code>	函数
多元表达式	<code>sp.lambdify((x, y), expr, 'numpy')</code>	函数
函数传入标量	<code>f_num(2)</code>	数值
函数传入数组	<code>f_num(arr)</code>	NumPy 数组
不指定模块	<code>sp.lambdify(x, expr)</code>	只支持普通 Python 标量/列表

常见错误

- 对 lambdify 生成的结果再次 diff / subs：它已经是数值函数，不是符号表达式；
- 没传 'numpy'，导致传入 NumPy 数组时报错或返回列表而非数组；
- 把 `sp.symbols('x y')` 返回的元组直接当成单符号传给 lambdify；
- 忘记先导入 numpy (`lambdify(..., 'numpy')` 运行时用到)。

拓展 / 跟练

1. 把 $f(x) = \sin(x)/x$ 用 lambdify 转成函数，计算 `np.linspace(0.01, 10, 5)` 处的值；
2. 用 lambdify 把 02 节解出的根/多项式转成数值函数，再用 `np.roots` 验证；
3. 对比循环 subs 与 lambdify 在 10000 个点上的耗时（用 time）。

2.4 用 dsolve 解常微分方程

2.4.1 一阶线性微分方程 $y' + y = x$

```
x = sp.symbols('x')
y = sp.Function('y')(x)
```

```
eq = sp.Eq(y.diff(x) + y, x)
print(sp.solve(eq, y))
Eq(y(x), C1*exp(-x) + x - 1)
```

2.4.2 二阶常系数齐次方程 $y'' - 2y' + y = 0$

```
x = sp.symbols('x')
y = sp.Function('y')(x)
eq = sp.Eq(y.diff(x, 2) - 2*y.diff(x) + y, 0)
print(sp.solve(eq, y))
Eq(y(x), (C1 + C2*x)*exp(x))
```

2.4.3 初值问题 (ics 参数)

```
x = sp.symbols('x')
y = sp.Function('y')(x)
eq = sp.Eq(y.diff(x) + y, x)
print(sp.solve(eq, y, ics={y.subs(x, 0): 1}))
Eq(y(x), x - 1 + 2*exp(-x))
```

2.4.4 微分方程组

$$\begin{cases} \frac{dx}{dt} = y \\ \frac{dy}{dt} = -x \end{cases}$$

```
t = sp.symbols('t')
xf = sp.Function('x')(t)
yf = sp.Function('y')(t)
eq1 = sp.Eq(xf.diff(t), yf)
eq2 = sp.Eq(yf.diff(t), -xf)
print(sp.solve([eq1, eq2], dict=True))
[Eq(x(t), C1*sin(t) + C2*cos(t)), Eq(y(t), C1*cos(t) - C2*sin(t))]
```

2.4.5 逻辑斯蒂模型 (含初值)

$$\frac{dx}{dt} = rx \left(1 - \frac{x}{K}\right), x(0) = x_0$$

```
import sympy as sp
x = sp.symbols('x', cls=sp.Function)
x0, t, r, K = sp.symbols('x0 t r K', real=True, positive=True)
sol = sp.solve(sp.Eq(x(t).diff(t), r*x(t)*(1 - x(t)/K)), ics={x(0): x0})
print(sol)
Eq(x(t), K*x0*exp(r*t)/((-K + x0)*(x0*exp(r*t)/(-K + x0) - 1)))
```

注: dsolve 求解的是解析解; 对无法解析求解的复杂方程, 应改用 SciPy 的 solve_ivp/odeint 做数值解 (详见后续章节)。

常见误区

1. **混淆 solve 与 solveset 的返回类型**: solve 返回列表, solveset 返回集合; 两者都不一定能给出闭式解。
2. **忘记 Eq 的两种写法**: `sp.Eq(x**2, 4)` 与直接传 `x**2 - 4` 等价, 但若写成 `[x**2, y**2]` 会被当成 “两个方程都等于 0”。
3. **把 lambdify 的结果当符号**: 它是数值函数, 输入是数字/数组, 输出是数值; 不要再对它调用 `diff`、`subs`。
4. **用 Matrix 做逐元素乘法**: SymPy 的 `A * B` 是矩阵乘法, 而 `A.dot(B)` 也做矩阵乘; 没有 NumPy 里的 `*` 逐元素语义。
5. **dsolve 的 ics 用错格式**: 初值应写成字典, 如 `{y.subs(x, 0): 1}` (`y` 需先定义为 `Function('y')(x)`)。
6. **以为 linsolve 总能给唯一解**: 无解返回 `EmptySet`, 无穷多解返回带参数的解集。

思考题

1. 为什么 `sp.solve([x**2 + y**2 - 5, x - y - 1], (x, y))` 会返回列表而不是字典? 它与 2.1.4 的线性方程组返回字典有何不同?
2. `lambdify` 的第三个参数为什么建议用 `'numpy'`? 不指定时对数组输入会怎样?
3. 用 `Matrix.solve` 与 `linsolve` 解同一个方程, 结果形式有何区别?
4. `dsolve` 返回的通解中 C_1 、 C_2 是什么? 如何用 `ics` 消去它们?

动手练习 (详见 lab)

- 用 `solve` 与 `solveset` 分别解 $x^3 - 6x^2 + 11x - 6 = 0$, 比较返回形式;
- 用 `linsolve` 解一个三元欠定方程组 (无穷多解) 并观察输出;
- 用 `lambdify` 把 $f(x, y) = x^2 + xy + y^2$ 转成可接受二维数组的函数;
- 用 `dsolve` 求 $y'' + 4y = 0$ 的通解并写出含 $\sin(2x)$ 、 $\cos(2x)$ 的形式。

延伸阅读

- SymPy 官方 Solvers 文档: [链接](#)
- SymPy 官方 Matrix 文档: [链接](#)
- SymPy 官方 lambdify 参考: [链接](#)
- SymPy 官方 ODE (dsolve) 文档: [链接](#)

03 综合案例：符号推导 → 数值验证 → 可视化

本章新增综合案例，把 01-02 的知识串起来：**符号积分 / 符号导数 + lamdify + matplotlib 可视化**。所有代码在 SymPy 1.13.1、NumPy 2.1.2、Matplotlib 3.10.1 环境运行通过；配图由 build/make_chap2_figures.py 生成。

案例 1：用符号积分求曲线下面积并数值验证

背景

求 $y = x^2$ 在 $[0, 1]$ 上与 x 轴围成的面积。用 SymPy 可精确求得 $\int_0^1 x^2 dx = \frac{1}{3}$ ，再用数值 (N/evalf) 验证，并画出图形。

代码

```
import sympy as sp
x = sp.symbols('x')
f = x**2
area = sp.integrate(f, (x, 0, 1))
print(area)          # 精确结果
print(sp.N(area))    # 数值近似

1/3
0.3333333333333333
```

可视化

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

import os
os.makedirs('images', exist_ok=True)

xs = np.linspace(0, 1, 300)
ys = xs**2
plt.figure(figsize=(5.4, 4.0))
plt.fill_between(xs, ys, color='#9ec7e8', alpha=0.55, label=f'S = int x^2 dx = {float(area)}')
plt.plot(xs, ys, color='#2f6fb3', lw=2.4, label='y = x^2')
plt.xlabel('x'); plt.ylabel('y')
plt.title('符号积分求面积并用数值验证', fontsize=11)
plt.legend(); plt.grid(alpha=0.25)
plt.savefig('images/sympy_area.png', bbox_inches='tight')
plt.show()
```

分析

- SymPy 给出精确的 $1/3$ ，而 NumPy 的 np.trapz/数值积分只能给出近似；

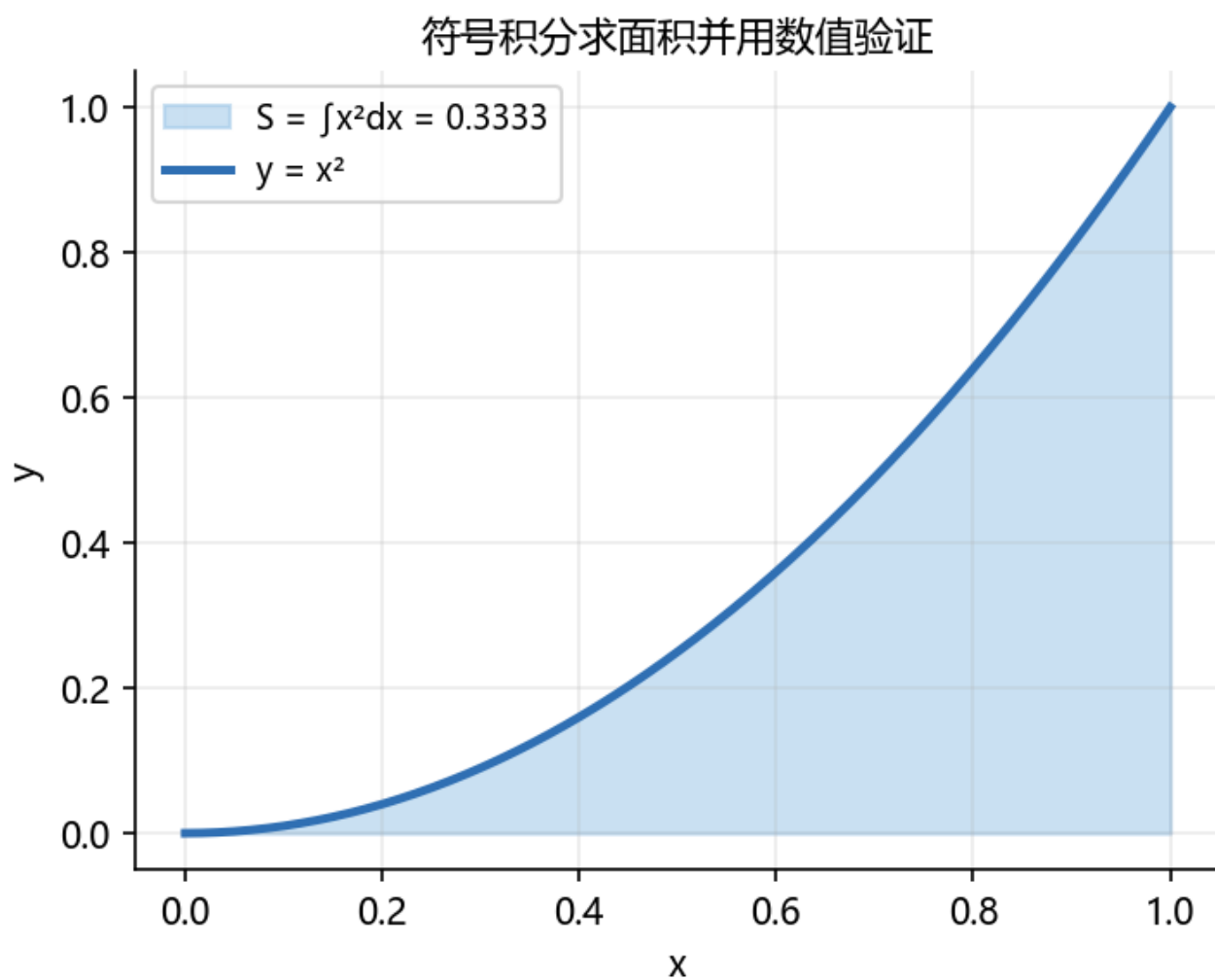


Figure 1: 符号积分求面积

- `sp.N(area)` 输出 0.333333333333333, 说明符号结果与数值结果一致;
- 这正体现 “先符号推导得到公式, 再数值计算/可视化” 的方法论。

拓展任务

1. 改求 $y = e^{-x}$ 在 $[0, 1]$ 上的面积, 并与 `scipy.integrate.quad` 比较;
2. 求曲线 $y = x^2$ 绕 x 轴旋转一周的旋转体体积: $\pi \int_0^1 x^4 dx$, 用 SymPy 验证;
3. 把 $[0, 1]$ 换成 $[a, b]$ 符号区间, 观察 SymPy 是否保留符号结果。

案例 2: 符号导数 → 切线方程 → 数值验证

背景

求 $y = \sin x$ 在 $x = 1$ 处的切线方程。先用 SymPy 求导得到 $\cos x$, 再求斜率 $\cos(1)$, 最后用 `lambdify` 画图, 并与数值差商验证。

代码

```
import sympy as sp
import numpy as np
x = sp.symbols('x')
f = sp.sin(x)
df = sp.diff(f, x)
print(df)    # cos(x)

a = 1
slope = sp.N(df.subs(x, a))
print(slope)    # cos(1) ≈ 0.540302305868140

tangent = sp.simplify(df.subs(x, a) * (x - a) + f.subs(x, a))
print(tangent)

# 数值差商验证导数
h = 1e-6
numeric_deriv = (sp.sin(a + h) - sp.sin(a - h)) / (2*h)
print(f"{float(sp.N(numeric_deriv)):.12f}")

cos(x)
0.540302305868140
(x - 1)*cos(1) + sin(1)
0.540302305840
```

可视化

```
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
```



```

import os
os.makedirs('images', exist_ok=True)

def tangent_num(v):
    return float(sp.N(tangent.subs(x, v)))

xn = np.linspace(-1, 3, 400)
plt.figure(figsize=(5.8, 4.0))
plt.plot(xn, np.sin(xn), color='#2f6fb3', lw=2.4, label='y = sin(x)')
plt.plot(xn, [tangent_num(v) for v in xn], color='#e07b39', lw=2, ls='--',
         label=f'切线 (斜率 ≈ {float(slope):.4f})')
plt.plot([a], [np.sin(a)], 'o', color='#c0392b', ms=7, label=f'切点 x={a}')
plt.axhline(0, color='#666', lw=0.7); plt.axvline(0, color='#666', lw=0.7)
plt.xlabel('x'); plt.ylabel('y')
plt.title('符号导数 → 切线方程', fontsize=11)
plt.legend(); plt.grid(alpha=0.25)
plt.savefig('images/sympy_tangent.png', bbox_inches='tight')
plt.show()

```

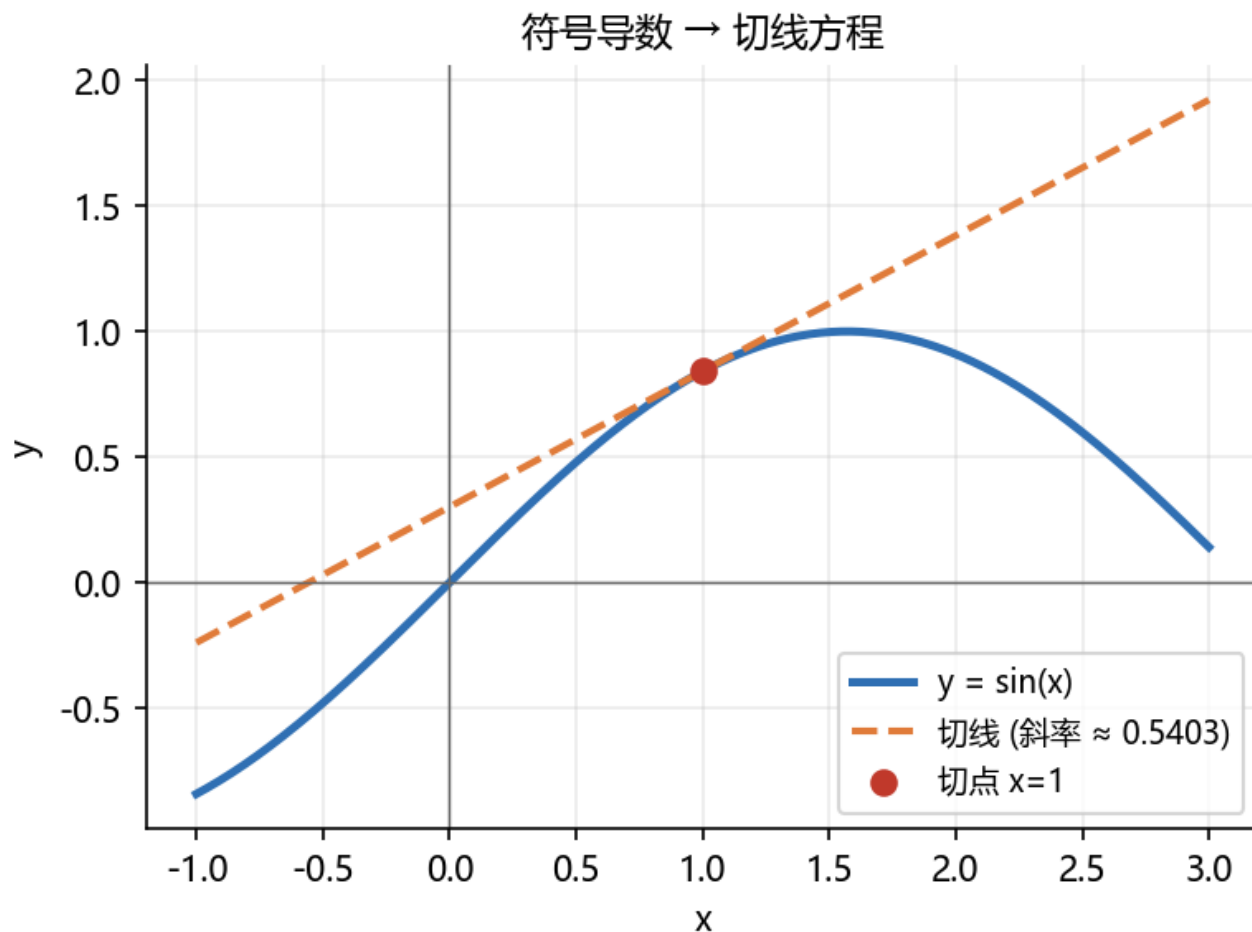


Figure 2: 符号导数与切线

分析

- 符号求导得到 $\cos x$, 代入 $x = 1$ 得斜率 ≈ 0.5403 ;
- 数值中心差商 ($h = 10^{-6}$) 与符号导数一致, 说明推导正确;
- 切线方程被 `simplify` 整理成 $(x-1)*\cos(1) + \sin(1)$, 可直接用于绘图/插值。

拓展任务

1. 用 `lambdify` 把 `tangent` 和 `f` 都转成数值函数, 再比较在 $x = 0.5, 1, 1.5$ 处的数值;
2. 求 $y = \ln(x)$ 在 $x = 2$ 的法线方程并绘图;
3. 把切线推广到二元函数 (用梯度向量/切平面概念)。

案例 3: 解方程并把根画在数轴上

背景

用 `solve` 求出多项式 $x^2 - 5x + 4 = 0$ 的根, 并把函数曲线与根标注在同一张图上, 直观理解 “方程的解” 就是 “函数与 x 轴交点”。

代码与输出

```
import sympy as sp
x = sp.symbols('x')
p = x**2 - 5*x + 4
roots = sorted(sp.solve(p, x))
print(roots)
print([float(sp.N(r)) for r in roots])

[1, 4]
[1.0, 4.0]
```

分析

- `solve` 精确返回整数根 1 和 4, 无需浮点迭代;
- 根的位置与曲线穿越 x 轴的位置一致, 图片直观地体现 “解方程 = 找零点”。

拓展任务

1. 改求 $x^3 - 6x^2 + 11x - 6 = 0$ 的三个根并画图;
2. 用 `solveset(..., domain=sp.S.Complexes)` 求解 $x^2 + 1 = 0$, 解释为什么实数域为空;
3. 用 `lambdify` 把 `p` 转成数值函数, 用 `np.roots` 验证数值根。

本章综合练习 (上机任务)

1. 用 SymPy 推导 $y = \sqrt{x}$ 在 $[0, 4]$ 上的面积, 用 `matplotlib` 绘制填充区域并标注精确值;
2. 求 $f(x) = x^3 - 3x$ 的极值点 (提示: 令 $f'(x) = 0$ 用 `solve`), 并画函数与极值点图;
3. 把案例 1-3 整理成一个 `.ipynb`, 写一份 200 字以内的结论, 说明 “符号解 vs 数值解” 各自的优势。

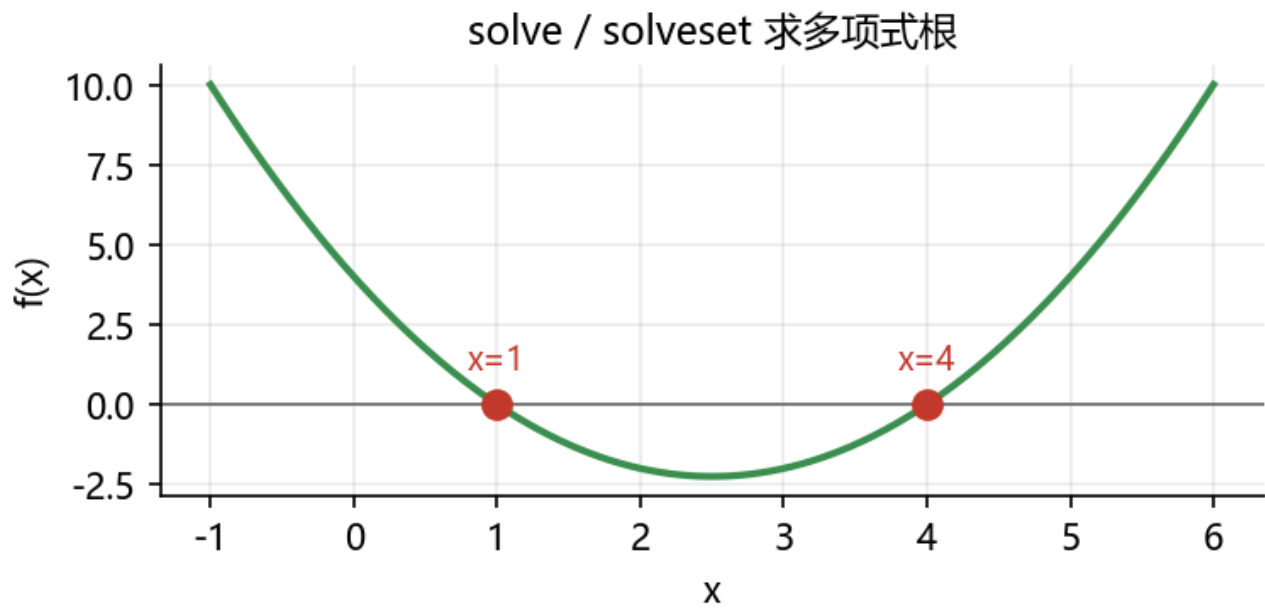


Figure 3: 多项式根可视化

延伸阅读

- SymPy 官方绘图 (Plotting) 模块: [链接](#)
- Matplotlib 填充区域: [链接](#)
- NumPy 数值求根 (np.roots): [链接](#)

04 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确的写法和性能技巧。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	<code>pi = 3.14159</code> 然后参与符号运算	用 <code>sp.pi</code> (符号 π)	浮点近似破坏精确性
2	<code>x + y == y + x</code> 判断表达式相等	<code>sp.simplify((x+y) - (y+x)) == 0</code> 或 <code>(x+y).equals(y+x)</code>	<code>==</code> 比较的是结构不是数学等价
3	<code>sp.sqrt(x**2)</code> 期望得到 <code>x</code>	<code>x = sp.Symbol('x', positive=True)</code>	若无假设, <code>sqrt(x**2)= x </code> 更严谨
4	用 <code>math.sin(x)</code> 而 <code>x</code> 是符号	用 <code>sp.sin(x)</code>	<code>math</code> 只能处理浮点数
5	以为 <code>lambdify</code> 返回符号表达式	它是数值函数, 输入输出都是数值/数组	符号 $\rightarrow$ 数值的桥梁, 不可再 <code>diff</code>
6	<code>A * B</code> 想要 NumPy 式的逐元素乘	SymPy Matrix 的 <code>*</code> 是矩阵乘; 逐元素需 <code>sp.Matrix(A.applyfunc)</code>	两种库的语义不同
7	<code>sp.solve(eq, x)</code> 忘记 <code>Eq</code>	可以写 <code>sp.solve(eq, x)</code> (<code>eq</code> 视为 <code>=0</code>) 或 <code>sp.solve(sp.Eq(f, 0), x)</code>	传入非 <code>Eq</code> 表达式默认等于 0
8	用 <code>dsolve(eq)</code> 但没定义 <code>Function</code>	<code>y = sp.Function('y')(x);</code> 初值用 <code>ics={y.subs(x,0): 1}</code>	未定义函数会让 <code>dsolve</code> 报错/无法识别
9	期望 <code>simplify</code> 万能	配合 <code>trigsimp</code> 、 <code>together</code> 、 <code>cancel</code> 、 <code>radsimp</code>	化简是启发式的
10	用 <code>solve</code> 求所有复数根当实数用	用 <code>solveset(..., domain=sp.S.Reals)</code> 或 <code>real_roots</code>	不同求解函数返回集不同

2. 原理级难点：符号 vs 数值

```
import sympy as sp
x = sp.Symbol('x')
expr = x**2 + 1          # 符号表达式
print(type(expr))        # <class 'sympy.core.add.Add'>

val = expr.subs(x, 2)    # 仍是符号 (整数 5)
print(val, type(val))

num = sp.N(expr.subs(x, 2)) # 数值
print(num, type(num))
```

```
f = sp.lambdify(x, expr, 'numpy')
print(f(2))           # 数值 5
```

要点：符号对象不参与浮点运算，只有 subs+N/evalf 或 lambdify 才会产生数值结果。

3. 求解函数的选型

场景	推荐函数	说明
一元方程	solve / solveset	solveset 更严谨
线性方程组	linsolve	可传方程列表或 (A, b)
非线性方程组	nonlinsolve / solve	结果可能含复数解
初等矩阵运算	Matrix + 方法	det、inv、eigenvals
符号 → 数值	lambdify	指定 'numpy'

4. 性能技巧

- **优先 lambdify 而不是逐点 subs/evalf**：对大量数值点，先转成 NumPy 函数再向量化计算，速度可提升百倍：

```
import sympy as sp, numpy as np
x = sp.symbols('x')
f = sp.lambdify(x, sp.sin(x)/x, 'numpy')
xs = np.linspace(0.001, 10, 100000)
ys = f(xs)           # 一次向量化，比循环快得多
```
- **simplify 要慎用**：可能很慢、甚至把结果变得更长；先试 cancel/together/trigsimp；
- **避免反复重建符号**：把符号与常用表达式定义一次，不要在循环里重复 symbols；
- **大矩阵用 sp.Matrix 的数值化**：若最终只需数值，考虑 sp.Matrix(...).evalf() 或转成 NumPy np.array(...) 再算；
- **nsimplify 做“反向”精确化**：把浮点近似还原为有理数/根式（如 sp.nsimplify(0.333333) → 1/3）。

5. 调试建议

1. 打印 type(expr)：确认是符号还是数值；
2. 用 sp.srepr(expr) 查看底层树状结构，定位“为什么没化简”；
3. expr.free_symbols 查看表达式含哪些符号；
4. sp.solve 无解时先试 solveset，再看 domain；
5. 符号求导/积分结果可疑时，用数值验证：sp.N(expr.subs(x, a)) 与手工/数值结果对比；
6. 复杂方程先 sp.simplify 或 sp.expand 再求。

6. 本章小结（自测）

- ☐ 能说明 symbols 与 Symbol 的区别；
- ☐ 能区分符号 π 与 math.pi；
- ☐ 能正确判断两个符号表达式是否数学等价；

- ☐ 能求极限、导数、积分并用数值验证;
- ☐ 能用 `solve/solveset/linsolve/nonlinsolve` 解对应问题;
- ☐ 会用 `lambdify` 把表达式转成数值函数;
- ☐ 会用 `dsolve` 解一/二阶微分方程并给定初值;
- ☐ 能用 `Matrix` 求特征值、雅可比矩阵。

延伸阅读

- SymPy 官方 Tutorial: [链接](#)
- SymPy 官方 FAQ: [链接](#)
- SymPy 官方性能/数值化建议: [链接](#)